

Pós-Graduação em Sistemas Computacionais Embebidos
Sistemas Operativos de Tempo-Real (PGSCE-SIOPTR)

Exercício individual

Duração: 1h:00m

Data: 30/05/2028

Número: 1203200

Nome: André Baltazar Pinto

Versão: A

- Responda somente a uma única opção por pergunta.
 - Escolha a opção mais completa e adequada conforme os conteúdos/discussão das aulas;
 - Caso se engane no preenchimento da tabela(s) abaixo, indique, por extenso a opção que considera correta.
- Se a resposta assinalada for incorreta sofrerá uma penalização de 1/3 da cotação da pergunta.
- Apenas as respostas assinaladas nesta página serão consideradas.
- É obrigatório a entrega de todas as folhas do exame.

		Opções			
		A	B	C	D
Perguntas	1	X			
	2			X	
	3		X		
	4			X	
	5		X		
	6		X		
	7		X		
	8			X	
	9		X		
	10	X			

		Opções			
		A	B	C	D
Perguntas	11	X	X		
	12	X			
	13	X			
	14				X
	15		X		
	16		X		
	17		X		
	18			X	
	19				X
	20			X	

Pós-Graduação em Sistemas Computacionais Embebidos
Sistemas Operativos de Tempo-Real (PGSCE-SIOPTR)

Exercício individual

Duração: 1h:00m

Versão: A

1. *General-Purpose Operating Systems* (GPOSs)

- (a) Usam políticas de escalonamento por forma a evitar o *starvation*;
- (b) São usados em sistemas de tempo-real;
- (c) Ambas opções anteriores;
- (d) Nenhuma das opções anteriores.

2. Num sistema operativo, uma interrupção:

- (a) É um evento que altera a sequência de execução das instruções;
- (b) Pode ser gerada por um processador;
- (c) Ambas opções anteriores;
- (d) Nenhuma das opções anteriores.

3. *Linux kernel build system*:

- (a) Usa símbolos de configuração para compilar condicionalmente o *kernel* do Linux;
- (b) Constrói o *object code* iterando de forma recursiva e descendente na estrutura de código fonte;
- (c) Ambas opções anteriores;
- (d) Nenhuma das opções anteriores.

4. No *kernel* do Linux:

- (a) Uma *System call* é identificada por um número;
- (b) Uma *System call* é identificada por um *interrupção*;
- (c) Ambas opções anteriores;
- (d) Nenhuma das opções anteriores.

5. No kernel do Linux, o escalonamento de tarefas:

- (a) É organizado por prioridades;
- (b) Usa classes de escalonamento para implementar políticas de escalonamento;
- (c) Ambas opções anteriores;
- (d) Nenhuma das opções anteriores.

6. O kernel do Linux:

- (a) Cria uma instância do tipo `struct rq` por CPU;
- (b) Cria uma instância do tipo `struct rq` para todos CPUs;
- (c) Ambas opções anteriores;
- (d) Nenhuma das opções anteriores.

7. No kernel do Linux, há uma `idle task`:

- (a) Para o sistema;
- (b) Escalonada de acordo com política de escalonamento identificada por `SCHED_IDLE`;
- (c) Ambas opções anteriores;
- (d) Nenhuma das opções anteriores.

8. No kernel do Linux, o `tick rate` é definido por `HZ`:

- (a) O `tick` é um interrupção temporal;
- (b) A cada `tick` a função `scheduler core` (`__schedule`) é invocada;
- (c) Ambas opções anteriores;
- (d) Nenhuma das opções anteriores.

9. No kernel do Linux, relativamente aos mecanismos de sincronização:

- (a) O valor de um semáforo pode ser alterado por qualquer processo/thread;
- (b) Somente a `thread` que é "owner", que fez o `lock`, do `mutex` é que pode libertar, fazer `unlock` do mesmo;
- (c) Ambas opções anteriores;
- (d) Nenhuma das opções anteriores.

10. No kernel do Linux, relativamente ao mecanismo de sincronização `spinlock`:

- (a) Uma tarefa que faça `lock` de um `spinlock` indisponível é colocado numa `waitqueue`;
- (b) Uma tarefa que faça `lock` de um `spinlock` indisponível é automaticamente removido do processador;
- (c) Ambas opções anteriores;
- (d) Nenhuma das opções anteriores.

Linux;
a de diretórios

11. No kernel do Linux, as operações com variáveis do tipo `atomic_t`:

- (a) São atômicas.
- (b) O valor contido nela por cada acesso não é limitado a 24 bits.
- (c) Ambas opções anteriores.
- (d) Nenhuma das opções anteriores.

(a) 4
(b) 04
(c) 04
(d) 04
12. 3

12. No kernel do Linux, as interrupções:

- (a) Podem ser desativadas.
- (b) Não podem ser desativadas.
- (c) Ambas opções anteriores.
- (d) Nenhuma das opções anteriores.

13. O kernel do Linux disponibiliza a implementação de árvores binárias do tipo Red-Black:

- (a) São adequadas para situações que impliquem rotações.
- (b) São auto-balanceadoras.
- (c) Ambas opções anteriores.
- (d) Nenhuma das opções anteriores.

14. No kernel do Linux, a macro `container_of (ptr, type, member)`:

- (a) Devolve o valor do member.
- (b) Devolve o valor do ptr.
- (c) Ambas opções anteriores.
- (d) Nenhuma das opções anteriores.

15. Loadable kernel Modules:

- (a) Podem ser inseridos no kernel em tempo de compilação.
- (b) Podem ser inseridos no kernel em tempo de execução.
- (c) Ambas opções anteriores.
- (d) Nenhuma das opções anteriores.

10. Processos e threads:

- (a) Quando uma tarefa é criada pela função `fork` os atributos `tgid` e `pid` associados à nova tarefa são diferentes da tarefa pai.
- (b) Quando uma tarefa é criada pela função `pthread_create` o atributo `tgid` é igual ao `tgid` da tarefa pai, mas o atributo `pid` são diferentes.
- (c) Ambas opções anteriores.
- (d) Nenhuma das opções anteriores.

1. No mecanismo de transações (dead) do kernel do Linux:

- (a) O processo de transações tem como pontos de partida e chegada o estado `STARTING`.
- (b) O processo de transações tem como pontos de partida e chegada o estado `STARTING`.
- (c) Analisa opções anteriores.
- (d) Nenhuma das opções anteriores.

18. No kernel do Linux:

- (a) A transação de base é iniciada por `trans_start` e termina por `trans_end`.
- (b) A transação de base é iniciada por `trans_start` e termina por `trans_end`.
- (c) Analisa opções anteriores.
- (d) Nenhuma das opções anteriores.

19. No kernel do Linux:

- (a) Para cada processo é criada uma instância de `struct` `task_struct`.
- (b) Para cada processo é criada uma instância de `struct` `task_struct`.
- (c) Analisa opções anteriores.
- (d) Nenhuma das opções anteriores.

20. No kernel do Linux:

- (a) Para cada processo é criada uma instância do tipo `struct` `task_struct`.
- (b) Para cada thread é criada uma instância do tipo `struct` `task_struct`.
- (c) Analisa opções anteriores.
- (d) Nenhuma das opções anteriores.

17. No mecanismo de concessões (bidding) do kernel do Linux:

- (a) O processo de concessões tem como ponto de partida o futuro estado do sistema.
- (b) O processo de concessões tem como ponto de partida o futuro estado do sistema.
- (c) Analisa opções anteriores.
- (d) Necessaria das opções anteriores.

18. No kernel do Linux:

- (a) A concessão da função `fcntl` cria o processo em execução para um processo.
- (b) A concessão `fcntl` cria o processo a função `fcntl`.
- (c) Analisa opções anteriores.
- (d) Necessaria das opções anteriores.

19. No kernel do Linux:

- (a) Para cada processo é criada uma instância de `struct`.
- (b) Para cada processo é criada uma instância de `struct`.
- (c) Analisa opções anteriores.
- (d) Necessaria das opções anteriores.

20. No kernel do Linux:

- (a) Para cada processo é criada uma instância do tipo `struct`.
- (b) Para cada thread é criada uma instância do tipo `struct`.
- (c) Analisa opções anteriores.
- (d) Necessaria das opções anteriores.